

P08

La atención es todo lo que necesitas

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin · 2017 · arXiv:1706.03762 · NeurIPS (NIPS) 2017

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P08 — Transformer · *Attention Is All You Need*

El paper que quitó la recurrencia del modelado de secuencias y, con ella, el último obstáculo para entrenar a gran escala. Casi todo lo que vino después es una rama de este bloque.

Nivel: L4 · Motor: `transformer` · Notebook: `P08_transformer.ipynb` · Miniaturas: T01–T08

1. Identificación

Campo	Valor
Título original	<i>Attention Is All You Need</i>
Autoría	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
Año	2017
Venue	arXiv:1706.03762 (junio) · NeurIPS 2017 (diciembre)
Fuente primaria	arXiv:1706.03762 · actas NeurIPS 2017
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Tras P07, la atención había resuelto el cuello de botella. Pero el modelo seguía siendo **recurrente**, y eso arrastraba dos costes:

- Cómputo secuencial.** El estado `ht` necesita `h{t-1}`. No se puede paralelizar sobre la longitud de la secuencia, por muchas GPU que haya. El entrenamiento sobre corpus grandes se vuelve el cuello de botella real.
- Camino largo entre posiciones.** La señal entre la posición 1 y la 100 atraviesa 99 pasos. Cada paso es una oportunidad para que el gradiente se degrade.

Las alternativas convolucionales (ByteNet, ConvS2S) reducían la secuencialidad pero el camino entre posiciones distantes seguía creciendo con la distancia: logarítmica o linealmente.

La pregunta del paper: **si la atención ya conecta cualquier par de posiciones directamente, ¿para qué sigue haciendo falta la recurrencia?**

3. Propuesta

Una arquitectura codificador–decodificador construida **solo** con:

- **self-attention multi-cabeza** (la única operación que mezcla información entre posiciones);
- **redes feed-forward por posición** (aplicadas de forma independiente a cada token);
- **conexiones residuales** y **layer normalization** alrededor de cada subcapa;
- **codificación posicional** sumada a los embeddings, porque la atención por sí sola es indiferente al orden.

Sin recurrencia y sin convolución. Todas las posiciones de una capa se computan a la vez, y el camino entre dos posiciones cualesquiera es de longitud constante.

4. Intuición sin fórmulas

Cada palabra pregunta al resto de la frase «¿quién de vosotros me importa?», recibe una respuesta ponderada y se actualiza con ella. Y todas las palabras lo hacen **a la vez**, no en fila. Un RNN es una fila de personas pasándose un mensaje al oído; el Transformer es una sala donde todos leen el mismo tablón simultáneamente.

Dónde deja de funcionar la analogía: en la sala, cada persona tiene que leer a todas las demás. Con n personas eso son n^2 lecturas. Por eso el contexto largo es caro, y por eso el Transformer no es «gratis»: cambió coste secuencial por coste cuadrático.

5. Matemática mínima

Atención por producto escalar escalado (ecuación 1)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- $Q \in \mathbb{R}^{n \times d_k}$ consultas · $K \in \mathbb{R}^{m \times d_k}$ claves · $V \in \mathbb{R}^{m \times d_v}$ valores.
- $Q \cdot K^T$ mide compatibilidad; **softmax** la convierte en pesos que suman 1; el producto por V mezcla la información.

Por qué $\sqrt{d_k}$: si las componentes de q y k son independientes con media 0 y varianza 1, entonces $q \cdot k$ tiene media 0 y **varianza d_k** . Sin normalizar, la magnitud de los scores crece con la dimensión, el softmax se satura y los gradientes se apagan.

Multi-cabeza

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O \\ \text{head}_i &= \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \\ d_k &= d_v = d_{\text{model}} / h \end{aligned}$$

Partir d_{model} en h subespacios permite atender a varios tipos de relación a la vez **sin aumentar el número de parámetros**, porque cada cabeza trabaja en una dimensión h veces menor.

Máscara causal

$$\text{score}_{ij} \leftarrow -\infty \quad \text{para } j > i \quad (\text{antes del softmax})$$

Se aplica **antes** del softmax, no después: poner a cero los pesos tras normalizar rompería la distribución.

Codificación posicional

```
PE(pos, 2i)   = sin( pos / 10000^{2i/d_model} )
PE(pos, 2i+1) = cos( pos / 10000^{2i/d_model} )
```

Se **suma** al embedding (no se concatena: eso cambiaría `d_model` y rompería la compatibilidad con las residuales).

Subcapa completa

```
salida = LayerNorm( x + Sublayer(x) )

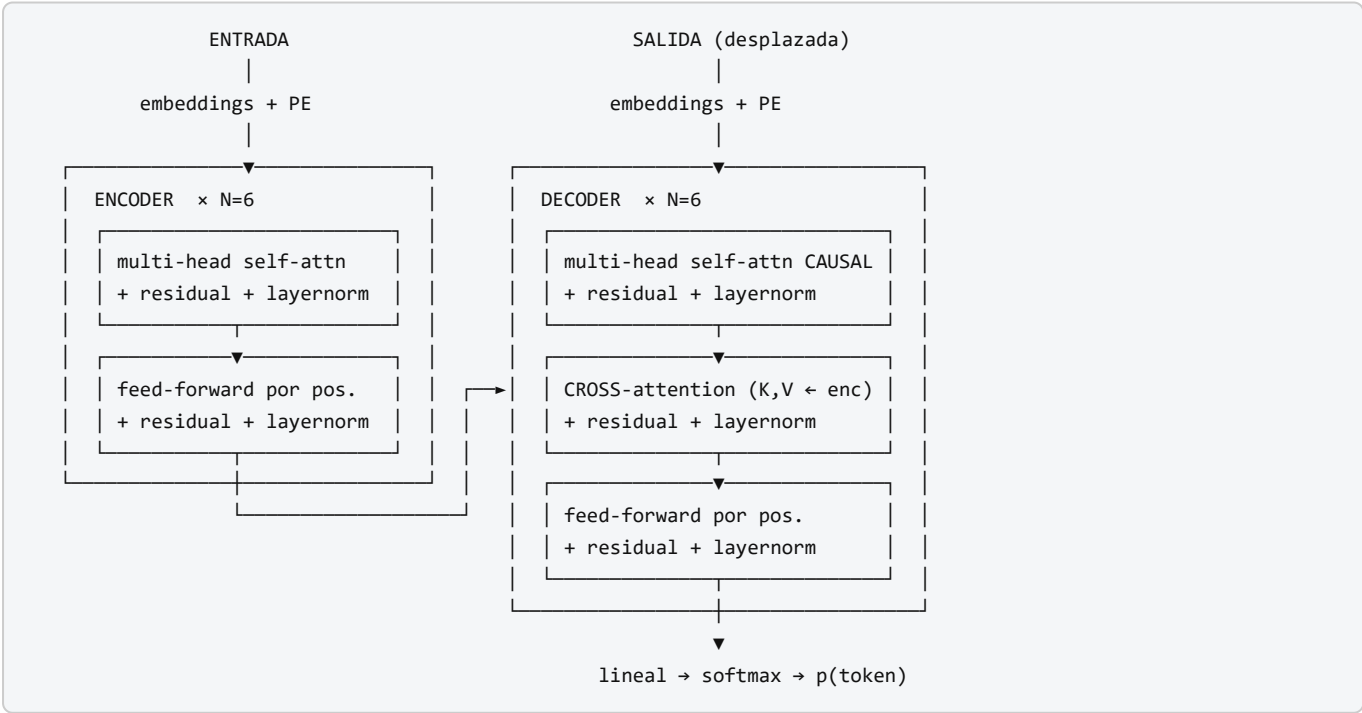
FFN(x) = max(0, x·W1 + b1) · W2 + b2      (misma FFN en todas las posiciones)
```

Complejidad (tabla 1 del paper)

Tipo de capa	Operaciones	Pasos secuenciales	Camino máximo
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrente	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutacional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

La atención sale más barata en operaciones cuando `n < d`, y siempre gana en paralelismo y en longitud de camino.

6. Arquitectura o flujo



Configuración base del paper: `N=6`, `d_model=512`, `h=8`, `d_k=d_v=64`, `d_ff=2048`, dropout `0,1`, *label smoothing* `0,1`, optimizador Adam con calentamiento de la tasa de aprendizaje.

7. Qué observar en el paper original

Dónde	Qué buscar
Figura 1	El diagrama de la arquitectura. Cuenta cuántas subcapas hay en el decoder frente al encoder: son tres contra dos.
Sección 3.2.1	La ecuación 1 y la justificación explícita de $\sqrt{d_k}$. Está escrita, no es folclore.
Sección 3.2.2	Por qué multi-cabeza y por qué <code>d_k = d_model/h</code> .
Sección 3.3	La FFN por posición. Es donde vive la mayoría de los parámetros del bloque.
Sección 3.5	Codificación posicional, y la comparación con una versión aprendida (resultados casi idénticos; eligen la sinusoidal por extrapolación a longitudes mayores).
Tabla 1	Complejidad por tipo de capa. Es la tabla que justifica todo el diseño.
Tabla 2	Resultados de traducción y coste de entrenamiento en FLOPs . Compara calidad y cómputo.
Tabla 3	Las ablaciones . La sección más honesta: qué pasa al variar <code>h</code> , <code>d_k</code> , el dropout o la codificación posicional.
Sección 5.4	Regularización: dropout y label smoothing. Detalles sin los cuales no se reproduce.

8. Evidencia y resultados

Traducción sobre **WMT 2014**, en dos pares: inglés→alemán e inglés→francés, medida en BLEU.

- El modelo grande reporta **28,4 BLEU en EN→DE** y **41,8 BLEU en EN→FR**, superando a los mejores modelos y conjuntos publicados hasta entonces.
- El modelo base alcanza resultados competitivos con **un coste de entrenamiento sustancialmente menor** que las alternativas; el modelo grande se entrenó en el orden de días sobre 8 GPU P100.
- Generaliza a otra tarea (análisis sintáctico de constituyentes en inglés) con cambios mínimos.

Los valores exactos por configuración, junto con los FLOPs de entrenamiento, están en la tabla 2 del artículo, y las ablaciones en la tabla 3. **Verificarlos allí antes de citarlos:** es habitual encontrar cifras mezcladas entre modelo base, modelo grande y conjuntos.

La miniatura de este eje aporta evidencia del mecanismo, no del resultado: sin la escala $\sqrt{d_k}$ la entropía media de la atención cae de `1,09` a `0,49` con `d_model=8`, y la matriz causal es exactamente triangular inferior con filas que suman 1.

9. Impacto

- **Toda la familia de modelos de lenguaje actual** descende de este bloque: BERT usa el encoder, GPT usa el decoder, T5 y BART usan ambos.
- La paralelización hizo económicamente viable el entrenamiento a escala, lo que a su vez hizo observables las **leyes de escalado** (Kaplan et al., 2020; Hoffmann et al., 2022).
- Desplazó a las CNN en visión (ViT, 2020) y llegó a audio, código, proteínas y series temporales.
- Convirtió el bloque en una **pieza de infraestructura**: se optimiza a nivel de kernel de GPU, de compilador y de hardware.

🚫 Qué no significa el título

Attention Is All You Need es una consigna, no un teorema. El propio modelo del paper necesita, además de la atención:

Pieza	Sin ella...
Feed-forward por posición	El bloque pierde la mayor parte de su capacidad; la atención sola es una mezcla lineal ponderada
Conexiones residuales	El gradiente no atraviesa 6 bloques apilados
Layer normalization	El entrenamiento se desestabiliza
Codificación posicional	El modelo es ciego al orden : «el perro muerde» y «muerde el perro» son idénticos
Escala $\sqrt{d_k}$	El softmax se satura y los gradientes se apagan
Label smoothing y warmup	No se reproducen los resultados reportados

Lo que el título afirma con precisión es que **no hacen falta recurrencia ni convolución**. Eso es mucho, y es distinto de «basta la atención».

10. Limitaciones

1. **Coste y memoria** $O(n^2)$ en la longitud de secuencia. Con $n = 128\ 000$, la matriz de atención de una sola cabeza y capa ocupa decenas de gigabytes en `float32`.
2. **Extrapolación limitada** a longitudes mucho mayores que las de entrenamiento, pese a la codificación sinusoidal.
3. **Hambriento de datos**. Sin sesgo inductivo de localidad (a diferencia de una CNN), necesita más ejemplos para aprender estructura.
4. **La atención no es explicación**, igual que en P07.
5. **Sin memoria persistente** entre secuencias: todo lo que hay es la ventana de contexto.
6. **Coste ecológico y económico** del entrenamiento, no discutido en el artículo.
7. **El resultado se demuestra en traducción**, no en «lenguaje» en general. La generalización posterior es un hecho histórico, no una afirmación del paper.

11. Errores comunes

Error	Corrección
«El Transformer solo usa atención»	Ver la tabla de la sección 9. Usa FFN, residuales, layer norm y codificación posicional.
«El Transformer inventó la atención»	La inventó Bahdanau et al. (2014). Este paper la generaliza y elimina el resto.
«Es más eficiente que un RNN»	Es más paralelizable siempre; más eficiente en operaciones solo si $n < d$.
«GPT usa el Transformer completo»	GPT usa solo el decoder , sin cross-attention. BERT usa solo el encoder.
«La codificación posicional se concatena»	Se suma . Concatenar cambiaría <code>d_model</code> y rompería las residuales.
«La máscara causal se aplica al softmax»	Se aplica antes , poniendo los scores a <code>-∞</code> .
« $\sqrt{d_k}$ es una constante de ajuste empírica»	El paper la justifica por la varianza del producto escalar (sección 3.2.1).
«El Transformer entiende el lenguaje»	Ganó en BLEU sobre WMT 2014. Todo lo demás es interpretación.

12. Relación con trabajos anteriores

- **P07 Attention (2014)** — el mecanismo que se generaliza.
- **P06 Seq2Seq (2014)** — el patrón codificador–decodificador que se conserva.
- **P04 AlexNet (2012)** y **ResNet (2015)** — profundidad y conexiones residuales.
- **Ba, Kiros y Hinton (2016)** — layer normalization. [arXiv:1607.06450](#)
- **Lin et al. (2017)**, **Cheng et al. (2016)** — self-attention previa en otras formulaciones.
- **ByteNet y ConvS2S (2016–2017)** — alternativas convolucionales a la recurrencia.

13. Relación con trabajos posteriores

- **P09 BERT (2018)** — rama encoder.
- **P10 GPT-3 (2020)** — rama decoder llevada a escala.
- **T5, BART (2019–2020)** — rama codificador–decodificador preentrenada.
- **ViT (2020)** — el mismo bloque aplicado a parches de imagen.
- **Kaplan et al. (2020)** — leyes de escalado. [arXiv:2001.08361](#)
- **Hoffmann et al. (2022)** — Chinchilla: cómputo óptimo entre datos y parámetros. [arXiv:2203.15556](#)
- **Variantes eficientes** (atención dispersa, lineal, con E/S optimizada) — todas atacan el $O(n^2)$ que este paper introdujo.

14. Notebook asociado

Este hito tiene **tratamiento especial**: una miniatura general más ocho que desmontan el bloque pieza por pieza.

Notebook	Qué aísla
P08_transformer.ipynb	Vista integrada: escala, máscara, multi-cabeza, PE, residual, complejidad
T01	Por qué había que quitar la recurrencia
T02	Q, K, V y la ecuación 1
T03	Softmax, escala $\sqrt{d_k}$ y saturación
T04	Self-attention y máscara causal
T05	Multi-head attention
T06	Codificación posicional
T07	Residual, layer norm y feed-forward
T08	Encoder-decoder, complejidad y qué no dice el título

Qué NO implementan: proyecciones W^Q , W^K , W^V aprendidas, entrenamiento, tokenización BPE, traducción ni evaluación BLEU. Son miniaturas del mecanismo, en Python estándar.

```
ai-evolution paper-lab P08 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación 1 y define Q , K , V , d_k sin mirar.
Explicar	Explica por qué la codificación posicional es imprescindible, usando el experimento de permutación de T06.
Aplicar	Ejecuta T04 y verifica que la masa sobre el futuro es exactamente 0 y que cada fila suma 1.
Analizar	Con $d_{model}=512$ y $h=8$, calcula parámetros de la atención multi-cabeza y de la FFN de un bloque. ¿Cuál domina?
Evaluar	Un artículo divulgativo titula «la atención es todo lo que necesitas: el resto sobra». Escribe una refutación de 5 líneas apoyada en la tabla 3 del paper.
Crear	Diseña una ablación propia: elimina la codificación posicional de un modelo de juguete y define qué tarea revelaría el fallo.

16. Autoevaluación

- ¿Por qué se divide por $\sqrt{d_k}$ y qué ocurre exactamente si no se hace?
- ¿Cuál es la diferencia entre self-attention y cross-attention, y dónde aparece cada una?
- ¿Por qué la máscara causal se aplica antes del softmax?
- ¿Por qué h cabezas de dimensión d/h no cuestan más parámetros que una de dimensión d ?
- ¿Por qué el modelo sería ciego al orden sin codificación posicional? Da un ejemplo concreto.
- ¿En qué régimen de n y d la atención hace más operaciones que la recurrencia?
- ¿Qué gana y qué paga el Transformer respecto a un RNN? Una frase para cada cosa.
- ¿Qué afirma exactamente el título y qué no afirma?

17. Respuestas esperadas

1. Porque $q \cdot k$ tiene varianza d_k cuando las componentes son independientes con varianza 1. Sin escalar, los scores crecen con la dimensión, el softmax se satura hacia un vector casi one-hot y el gradiente de las posiciones no seleccionadas se anula.
2. En self-attention, Q , K y V proceden de la misma secuencia (encoder, y decoder con máscara). En cross-attention, Q viene del decoder y K , V del encoder: es el puente entre ambos.
3. Porque poner pesos a cero después de normalizar deja una distribución que ya no suma 1. Con $-\infty$ antes del softmax, el peso resultante es exactamente 0 y la fila sigue sumando 1.
4. Porque cada cabeza opera en d_{model}/h dimensiones. El total de las proyecciones concatenadas es el mismo que el de una única proyección de dimensión d_{model} .
5. Porque la atención es equivariante a permutaciones: reordenar la entrada solo reordena la salida. «El perro muerde al cartero» y «el cartero muerde al perro» producirían el mismo conjunto de representaciones.
6. Cuando $n > d$: $n^2 \cdot d > n \cdot d^2$ equivale a $n > d$. Con $d = 512$, a partir de secuencias de más de 512 tokens.
7. **Gana:** paralelismo total dentro de la capa y camino de longitud 1 entre cualquier par de posiciones.
Paga: coste y memoria cuadráticos en la longitud.
8. Afirma que **no hacen falta recurrencia ni convolución**. No afirma que baste la atención: el propio modelo lleva FFN, residuales, layer norm y codificación posicional.

18. Fuentes primarias

- Vaswani, A. et al. (2017). *Attention Is All You Need*. **NeurIPS 2017**. arXiv:1706.03762 · actas · consultado 2026-08-16.
- Ba, J. L., Kiros, J. R. y Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450 · consultado 2026-08-16.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361 · consultado 2026-08-16.
- Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv:2203.15556 · consultado 2026-08-16.



Evaluación — P08 · La atención es todo lo que necesitas

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Attention Is All You Need* (2017, arXiv:1706.03762 · NeurIPS (NIPS) 2017) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P08_transformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).